

retained because the code is compatible with both Raspberry Pi 4 and Raspberry Pi 5 when all required Python modules are installed.

Circuit diagram

Fig. 6 shows the system's circuit diagram. The design is based on a Raspberry Pi 5 controlling two servo motors, pan and tilt, through an I²C-based PWM servo driver (PCA9685). Communication between the Raspberry Pi and the PCA9685 occurs over I2C interface using I2C pins of Raspberry Pi.

A separate SMPS power supply provides a stable 5V voltage to drive the servo motors, as servos require higher current than the Raspberry Pi can safely supply. The ground connections of the SMPS, PCA9685, and Raspberry Pi are tied together to maintain a common reference.

The PCA9685 generates accurate PWM signals for the pan and tilt servos, enabling smooth, precise angular motion. Control logic, such as face tracking, is handled by the Raspberry Pi, while PWM generation is offloaded to the PCA9685 to avoid overloading the GPIO pins.

Testing. Open a Linux terminal in the directory containing the code, then run the `facetracking.py` script to begin testing.

Performance analysis. Here is a core performance analysis of both Raspberry Pi 5 setups, with and without the Hailo accelerator.

Raspberry Pi 5 with Hailo-8L. Pairing the Raspberry Pi 5 with the Hailo-8L AI accelerator significantly enhances face-tracking performance.

Model used: `yolov5s_personface_h8l.hef` (Hailo-8L-compiled format)

FPS is consistently ≥ 30 with multiple faces, and it increases to 60+ FPS when tracking a single face.

Tracking is extremely fast and highly responsive.

Faces are locked onto almost instantly.

With multiple faces, slight jitter may occur, but the system stabilises quickly.

Tracking speed is so high that the pan-tilt base may require mechanical reinforcement to withstand rapid movements.

Raspberry Pi 5 without Hailo-8L. Performance remains stable and predictable, but is slower in comparison. Faces are accurately tracked, although response time is lower than with the Hailo-8L accelerator.

Servo tuning (`servo.py`). Servo movement can be manually tuned using the following command:

```
$> python servo.py servo_no angle_to_move
```

Example:

```
$> python servo.py 0 120 # Positions
the PAN servo to 120 degrees
```

EFY

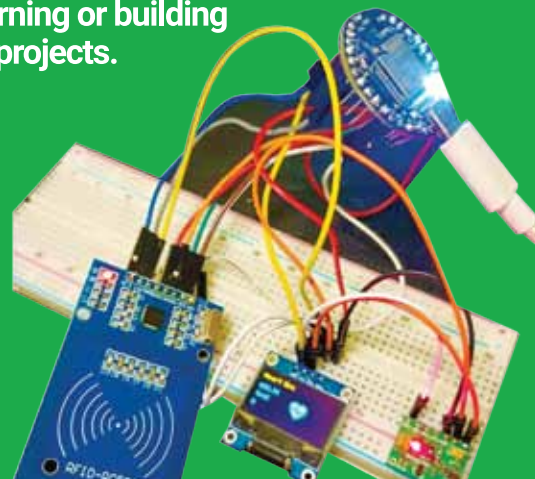


Somnath Bera is an electronics enthusiast. He is a freelancer and has contributed several articles across the globe.

DIY KIT: NFC, RFID, IoT & Embedded Systems

Want to learn and master NFC and RFID technology along with an IoT-enabled embedded system?

This kit includes all the essentials to start learning or building RFID/NFC projects.



POWERED BY



IndusBoard
COIN

Super small sized development board that enables you to create solutions for many use-cases where small size is super important.



SPECIAL INTRO PRICE

₹1,999 (Inclusive of GST)

KNOW MORE / BUY NOW: efy.co.in/nfcrid

Developed by the Electronics For You team, for you.

DEALERS: • Om Technology Centre, Bengaluru (Mob: +919845400864)
• Bharathi Electronics: Coimbatore (Mob: +919952228618)